

Hair Cutting Robots

Lecture 2: Rust and Simulator Setup

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

May 2026

The Hong Kong Polytechnic University

1

Installing Rust

What is Rust

A Compiler Language

Rust is a programming language which have both lower-level language's efficacy and higher-level language's feature.

GC? Ownership!

Different from lower-level language we need to manually operate on memory which causes potential bugs or like higher-level language using GC (Garbage collection) which contributes to runtime-overhead, rust introduce an ownership model to make sure programs are safe and fast.

Introduced in Linux

Rust is introduced as Linux's important component. Such as SudoRS, KernelRS.

<https://github.com/trifectatechfoundation/sudo-rs>

<https://docs.kernel.org/rust/index.html>

Why we learn Rust

Compiler is REALLY strong

Compilers can identify most of errors but not leaving them to runtime. It can detect the following errors:

- Unsafe memory usage (such as double free)
- Accessing undefined fields/functions
- And so on

Zero Runtime Overhead

Compiled in advance, what you write is what the program cost.
No hidden usage (Such as background GC).

Emerging chance

Rust is introduced as Linux as discussed before.

Compiler Detected Errors

Mutable and Not-accessed value warning

```
let mut a: i32 = 5;    // value assigned to 'a' is never read - maybe overwritten
a = 10;
println!("The value of a is: {}", a);
```

```
let a: i32 = 15;        // value assigned to 'a' is never read - maybe overwritten
a = 10;                // cannot assign twice to immutable variable 'a'
println!("The value of a is: {}", a);
```

Type checking

```
let a: String = String::from("Some String Pointer");
a = "Another String Pointer"; // mismatched types - expected 'String'
println!("The value of a is: {}", a);
```

RustUp

What is RustUp

The official Rust toolchain installer and version manager. It installs and manages multiple Rust versions and associated tools.

What you get

- `rustc` — the Rust compiler
- `cargo` — package manager & build tool
- `rustfmt` — code formatter
- `clippy` — linter

Install

MacOS / Linux

1. Run `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`
2. Restart your terminal or run: `source $HOME/.cargo/env`

Windows

1. Download RustUp installer on `rustup.rs`
2. Double click and follow the prompts.
3. If you don't have C++ environment, you may need them from `visualstudio.microsoft.com`

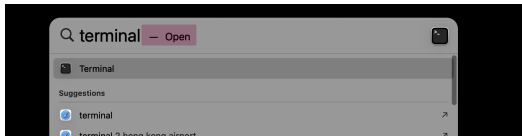
MacOS CLI

Open the Terminal APP

If you are using MAC, press `Cmd + Space` and type "Terminal". Then press `Enter` to open the terminal. (See the figure left below)

Typing the Command

When you go into the terminal, you will see a command prompt. Click on the window and type the exact command provided. Then press `Enter` to execute the command. (See the figure right below)



```
aa@MacBook-Air-3 Archive 3 % curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Setting PATH on Linux / MacOS

If `rustc` is not found after installation, the `cargo bin` directory is not on your `PATH` yet.

Quick check

```
echo $PATH  
ls $HOME/.cargo/bin
```

Add it for the current shell

```
source $HOME/.cargo/env
```

Add it permanently

Append to your shell config (`~/.zshrc` on macOS, `~/.bashrc` on Linux):

```
export PATH="$HOME/.cargo/bin:$PATH"
```

Verify

In CLI check

`rustc --version`, If you install successfully you will see the output of the current rust version.

If you are not successful, you may need to check whether the path environment is correctly set.

Create a new Rust Project!

Cargo is one component from the Rust toolchain. Run `cargo new <project_name>` to create a project!

Windows? Switch to WSL

Supported environments

In this course we only help with problems on **macOS** and **Debian-based Linux** (Debian / Ubuntu).

If you are on Windows

If anything breaks on native Windows, please switch to **WSL** (Windows Subsystem for Linux) and continue from there.

Native Windows issues are your own responsibility to fix.

Installing WSL

Install WSL (Ubuntu)

Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu
```

After installation

Reopen the terminal, launch Ubuntu, and redo the Rust installation steps inside the Linux shell.

Using WSL for Webpages

Visiting webpages from WSL

Use `curl` from **inside the Linux shell**, not from Windows PowerShell or CMD.

```
curl https://sh.rustup.rs
```

This ensures the request is made from the Linux environment, so downloaded files and installers land in the Linux file system where Rust will run.



Get Started with Rust

Your First Cargo Project

Create a project

```
cargo new hello_world  
cd hello_world
```

What was created

- `Cargo.toml` — the project manifest (name, version, dependencies)
- `src/main.rs` — the program entry point

Run it

```
cargo run
```

Your First Cargo Project (Cont'd)

Expected output

The first two lines are cargo telling you it has compiled the project. The last line is what your program actually printed.

```
Compiling hello_world v0.1.0 (/private/tmp/hello_world)
Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.68s
Running 'target/debug/hello_world'
Hello, world!
```

The path on the first line will differ on your machine — it is wherever you ran `cargo new`.

Reading main.rs

The generated program

```
fn main() {  
    println!("Hello, world!");  
}
```

Two new pieces

- `fn main()` — every Rust program starts here.
- `println!(...)` — a macro that prints a line. The `!` marks it as a macro, not a function.

Basic Arithmetic

Edit src/main.rs

```
fn main() {  
    let sum = 3 + 4;  
    println!("3 + 4 = {}", sum);  
}
```

What is new

- `let sum = 3 + 4;` — bind the value 7 to a name.
- `{}` inside the string is a placeholder; the value of `sum` fills it in.

Then run it again with `cargo run`.

Basic Arithmetic (Cont'd)

Expected output

Cargo recompiles your changed source, then runs the binary again. The last line is the new `println!` from your modified program.

```
Compiling hello_world v0.1.0 (/private/tmp/hello_world)
Finished 'dev' profile [unoptimized + debuginfo] target(s) in 0.14s
Running 'target/debug/hello_world'
3 + 4 = 7
```

The compile time is shorter than the first run — cargo caches results and only rebuilds what changed.



Get Started with Hotaru

Install the Hotaru CLI

Install

```
cargo install hotaru
```

This downloads, compiles, and installs the hotaru command-line tool.

Verify

```
hotaru version
```

Expected output

```
Hotaru version: 0.8.1
```

Scaffold a Web Project

Create the project

```
hotaru new web_hello  
cd web_hello
```

What was created

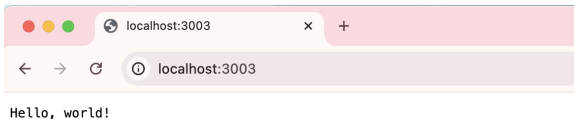
- Cargo.toml — already wired with the hotaru dependency
- src/main.rs — a tiny server returning “Hello, world!”
- build.rs — supporting build script
- templates/ — where HTML files will live (empty for now)

Run the Server

Start the server

```
cargo run
```

The server is now listening on 127.0.0.1:3003. Open `http://localhost:3003` in a browser:



Stopping the Server

Send the stop signal

In the terminal where `cargo run` is running, press:

`Ctrl + C`

The server shuts down and you get your shell prompt back.

Why we need to stop

Only one program can use port 3003 at a time. If you forget to stop the old server, the next `cargo run` will fail with “address already in use”.

The browser

The page in the browser stops updating — a refresh will fail until you start the server again.

Make it Yours

Open `src/main.rs` and find

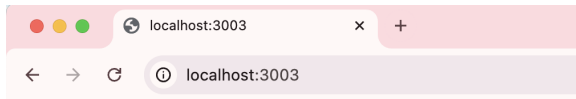
```
text_response("Hello, world!")
```

Change the string to anything you like

```
text_response("This is Hoshikawa Akari's blog!")
```

Try it

Restart the server (`Ctrl + C`, then `cargo run`) and refresh the browser.



This is Hoshikawa Akari's blog!

Rendering an HTML Template

Create templates/hello.html

Substitute values with `-[ var ]-`:

```
<h1>Hello, -[ name ]-!</h1>
```

Read the name from the URL

```
endpoint!{  
  APP.url("/<name>"),  
  pub home <HTTP> {  
    let name = req.param("name").unwrap_or_default();  
    akari_render!("hello.html", name = name)  
  }  
}
```

Rendering an HTML Template (Cont'd)

Try different names

Whatever follows the slash becomes the `name` parameter:

- `http://localhost:3003/Alice` — shows “Hello, Alice!”
- `http://localhost:3003/Bob` — shows “Hello, Bob!”

Server response for `/Alice`

```
<h1>Hello, Alice!</h1>
```

Returning JSON

Add a second endpoint

```
endpoint!{  
  APP.url("/api"),  
  pub api <HTTP> {  
    akari_json!({ status: "ok", count: 42 })  
  }  
}
```

Visit `http://localhost:3003/api`

The server replies with Content-Type: `application/json`:

```
{"status": "ok", "count": 42}
```



Hair Cutting Robot Simulator

Simulator Repo

Clone the repo

Run `git clone https://github.com/fds-pmine/hcr-simulator` to get the simulator code.

What is inside

1. `haircutting_core` - The core library for the hair cutting robot simulator, containing the main logic and algorithms.
2. `haircutting_web` - The web server component that provides a user interface for interacting with the simulator.

Running the Simulator

The web server is written in Rust using the Hotaru framework. To run the server, navigate to the `haircutting_web` directory and execute the following command:

```
cargo run -p haircutting_web
```

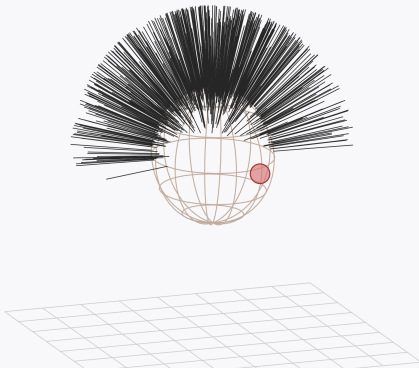
You should see output indicating that the server is running and listening on a specific port (e.g., `127.0.0.1:3003`).

Accessing the Simulator

Haircut Simulator — hotaru-served The simulation is paused until you press the button. Movement is hard-coded in Rust (MovementScript impl).

Start cutting paused

```
tick 0  
intact 650 · cut 0 · debris 0  
cutting false
```



Components of the Simulator

1. The circle in the center represents the head.
2. The black needle like lines represent the hair. (Notice since we didn't run Physics yet, the hair is just sticking out of the head instead of falling down.)
3. The red dot represents the hair cutting robot.
4. The start cutting button starts the physics simulation as well as the robot's movement.

How the Cutter is Programmed

The cutter's movement is controlled by a function called `tick`, found in `src/script.rs` of the `haircutting_web` project. The simulator calls this function 30 times per second, and each call decides where the cutter should be next.

Inside the function, a variable t counts how many ticks have happened since you pressed *Start cutting* ($t = 0, 1, 2, \dots$). Our program splits the haircut into three stages based on the value of t :

1. **Approach** — move toward the head with the cutter off.
2. **Activate** — turn the cutter on.
3. **Sweep** — slide the cutter across the head, trimming hair.

Stage 1 — Approach

When: $0 \leq t < 42$ (the first 42 ticks, about 1.4 seconds).

What: the cutter takes one small forward step every tick. The cutter is OFF, so no hair is trimmed yet — it is simply travelling toward the head.

How far: each step moves the cutter 0.04 units forward. After 42 ticks it has moved $42 \times 0.04 = 1.68$ units, which is enough to be positioned just above the head.

Stage 2 — Activate the Cutter

When: exactly at $t = 42$.

What: we send the cutter a single command to switch ON. From this moment on, any hair that comes within the cutter's reach is trimmed.

Note: this is a one-time switch. The cutter stays on for the rest of the program; we never turn it off again.

Stage 3 — Sweep

When: $42 \leq t \leq 242$ (the next 200 ticks, about 6.6 seconds).

What: the cutter slides across the top of the head, from the left ($x = -1$) to the right ($x = +1$), while the height stays the same ($z = 2$).

Formula. Let $p = \frac{t - 42}{200}$. The cutter is positioned at

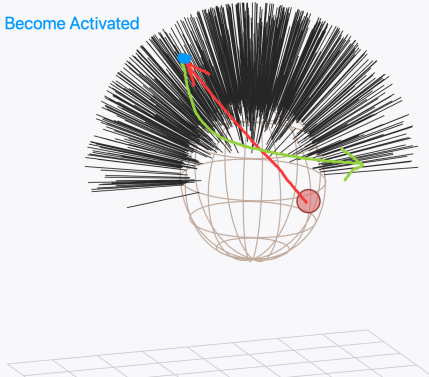
$$x = -1 + 2p^2, \quad z = 2.$$

Because we use p^2 instead of p , the cutter accelerates — it moves slowly through the left side (trimming more there) and quickly through the right.

Expected Path

```
tick 0  
intact 650 · cut 0 · debris 0  
cutting false
```

Become Activated



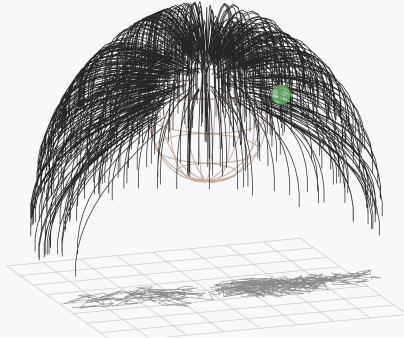
Just Start Running

```
tick 57  
intact 648 · cut 2 · debris 2  
cutting true
```



After Cutting

```
tick 2180  
intact 597 · cut 53 · debris 60  
cutting true
```



Key Features

1. The cutter will never go into the head.
2. After the hair is cut, there are debris left on the ground.
3. Physics will be simulated only if the program is running.
4. When the cutter shows red, it will not cut the hairs.
5. When the cutter shows green, it will cut the hairs.